МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Комп’ютерний практикум №6

з курсу «Основи розробки програмного забезпечення на платформі Microsoft.NET»

на тему: «Шаблони проектування. Поведінкові шаблони»

Викладач:

Ліщук К.І.

Виконав:

студент групи ІП-44

Комін Іван

Київ - 2026

Комп‘ютерний практикум № 6. Шаблони проектування. Поведінкові шаблони

Мета: - ознайомитися з основними шаблонами проектування, навчитися застосовувати їх при проектуванні і розробці ПЗ.

Постановка задачі комп‘ютерного практикуму № 6

При виконанні комп‘ютерного практикуму необхідно виконати наступні дії:

1) Вивчити структурні патерни. Знати загальну характеристику та призначення кожного з них, особливості реалізації кожного з породжуючих патернів та випадки їх застосування.

2) Реалізувати задачу згідно варіанту, запропонованого нижче у вигляді консольного застосування на мові C#. Розробити інтерфейси та класи з застосування одного або декількох патернів. Повністю реалізувати методи, пов‘язані з реалізацією обраного патерну.

3) Повністю описати архітектуру проекту (призначення методів та класів), особливості реалізації обраного патерну. Для кожного патерну необхідно вказати основні класи та їх призначення,

4) Навести UML-діаграму класів

5) Звіт повинен містити:

a. обґрунтування обраного патерну (чому саме він);

b. опис архітектури проекту (призначення методів та класів);

c. UML-діаграму класів

d. особливості реалізації обраного патерну

e. текст програмного коду

f. скріншоти результатів виконання.

Варіант індивідуальних завдань:

8) Реалізувати прототип симулятору бою, де бойові одиниці змінюють стан (атака, оборона, відступ, переслідування) залежно від подій на полі бою.

1. Обґрунтування обраного патерну

Для реалізації симулятора бою, де бойові одиниці змінюють свою поведінку залежно від обставин, обрано поведінковий патерн **State (Стан)**.

Чому саме цей патерн:

- **Динамічна зміна поведінки:** Юніт повинен кардинально змінювати свої дії (атакувати, тікати або наздоганяти) залежно від рівня здоров'я та стану ворога.
- **Усунення складних розгалужень:** Патерн дозволяє уникнути громіздких конструкцій if-else або switch всередині основного класу юніта, розподіляючи логіку за окремими класами станів.
- **Принцип єдиної відповідальності:** Кожен клас стану відповідає лише за один тип бойової поведінки, що полегшує тестування та підтримку коду.
- **Масштабованість:** Додавання нових станів (наприклад, "Патрулювання" або "Засідка") не потребує зміни існуючого коду інших станів.

2. Опис архітектури проєкту

Архітектура побудована на взаємодії контексту (юніта) та набору станів, що реалізують спільний інтерфейс.

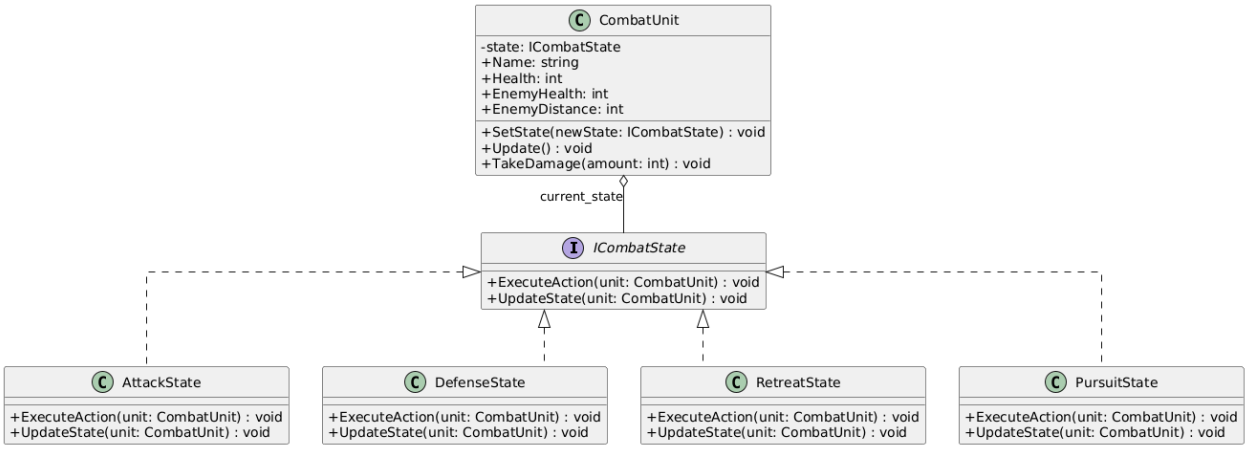
Основні класи та їх призначення:

- **CombatUnit (Context):** Головний клас бойової одиниці. Він зберігає поточні показники (Health, EnemyHealth, Distance) та посилання на об'єкт поточного стану. Всі запити на виконання дій він делегує цьому стану.
- **ICombatState (State Interface):** Інтерфейс, що визначає методи, які мають реалізувати всі бойові режими: ExecuteAction (виконання дії) та UpdateState (перевірка умов для зміни стану).
- **AttackState (Concrete State):** Реалізує логіку активного нападу. Перемикає юніта на відступ при низькому здоров'ї або на переслідування, якщо ворог слабшає.
- **DefenseState (Concrete State):** Реалізує захисну тактику з невеликою регенерацією. Дозволяє повернутися до атаки, якщо ворог далеко.
- **RetreatState (Concrete State):** Логіка відступу для збереження життя. Збільшує дистанцію до ворога.
- **PursuitState (Concrete State):** Стан переслідування пораненого ворога. Скорочує дистанцію та наносить фінальні удари для перемоги.

3. Особливості реалізації обраного патерну

- **Механізм делегування:** Метод Update() у класі CombatUnit просто викликає методи поточної стратегії поведінки, не знаючи, яка саме логіка там закладена.
- **Самостійні переходи:** Стани самі ініціюють зміну режиму через метод SetState контексту, що дозволяє створювати складні ланцюжки поведінки (наприклад, *Атака -> Переслідування -> Перемога -> Оборона*).
- **Інкапсуляція умов:** Всі магічні числа (пороги здоров'я для відступу тощо) приховані всередині відповідних класів станів, що відповідає принципу інкапсуляції.

UML-діаграма класів



Текст програмного коду

```
public interface ICombatState
{
    void ExecuteAction(CombatUnit unit);
    void UpdateState(CombatUnit unit);
}

public class CombatUnit
{
    private ICombatState _state;
    public string Name { get; }
    public int Health { get; set; } = 100;
    public int EnemyDistance { get; set; } = 50;
    public int EnemyHealth { get; set; } = 100;
```

```
    public CombatUnit(string name, ICombatState initialState)
    {
        Name = name;
        _state = initialState;
        Console.WriteLine($"[System]: {Name} initialized in
{initialState.GetType().Name}.");
    }

    public void SetState(ICombatState state)
    {
        Console.WriteLine($"[State Change]: {Name} transitions to
{state.GetType().Name}.");
        _state = state;
    }

    public void Update()
    {
        _state.ExecuteAction(this);
        _state.UpdateState(this);
    }

    public void TakeDamage(int amount)
    {
        Health -= amount;
        Console.WriteLine($"[Event]: {Name} took {amount} damage! Current
HP: {Health}");
    }
}

// --- Concrete States ---

public class AttackState : ICombatState
{
    public void ExecuteAction(CombatUnit unit)
    {
        Console.WriteLine($"-> {unit.Name} is attacking fiercely! (Deals 15
DMG)");
        unit.EnemyHealth -= 15;
    }

    public void UpdateState(CombatUnit unit)
    {
        if (unit.Health < 25)
            unit.SetState(new RetreatState());
        else if (unit.EnemyHealth < 30)
```

```
            unit.SetState(new PursuitState());
    }
}

public class DefenseState : ICombatState
{
    public void ExecuteAction(CombatUnit unit)
    {
        Console.WriteLine($"-> {unit.Name} is in defensive stance,
regenerating slightly.");
        unit.Health += 5;
    }

    public void UpdateState(CombatUnit unit)
    {
        if (unit.EnemyDistance > 100)
            unit.SetState(new AttackState());
        else if (unit.Health < 15)
            unit.SetState(new RetreatState());
    }
}

public class RetreatState : ICombatState
{
    public void ExecuteAction(CombatUnit unit)
    {
        Console.WriteLine($"-> {unit.Name} is retreating to save their
life!");
        unit.EnemyDistance += 20;
    }

    public void UpdateState(CombatUnit unit)
    {
        if (unit.Health > 50)
            unit.SetState(new DefenseState());
    }
}

public class PursuitState : ICombatState
{
    public void ExecuteAction(CombatUnit unit)
    {
        Console.WriteLine($"-> {unit.Name} is pursuing the weakened
enemy!");
        unit.EnemyDistance -= 15;
```

```
    }

    public void UpdateState(CombatUnit unit)
    {
        if (unit.EnemyHealth <= 0)
        {
            Console.WriteLine($"[Victory]: {unit.Name} defeated the
enemy!");
            unit.SetState(new DefenseState());
        }
        else if (unit.Health < 10)
            unit.SetState(new RetreatState());
    }
}

class Program
{
    static void Main()
    {
        var soldier = new CombatUnit("Infantryman", new AttackState());

        for (int i = 1; i <= 6; i++)
        {
            Console.WriteLine($"\n--- Round {i} ---");
            soldier.Update();

            if (i == 2)
                soldier.TakeDamage(80);
            if (i == 4)
                soldier.Health += 40;
        }
    }
}
```

Скріншоти результатів виконання

```
❯ /home/ivkins/.vscode/extensions/ms-dotnettools.csharp-2.130
[System]: Infantryman initialized in AttackState.

--- Round 1 ---
-> Infantryman is attacking fiercely! (Deals 15 DMG)

--- Round 2 ---
-> Infantryman is attacking fiercely! (Deals 15 DMG)
[Event]: Infantryman took 80 damage! Current HP: 20

--- Round 3 ---
-> Infantryman is attacking fiercely! (Deals 15 DMG)
[State Change]: Infantryman transitions to RetreatState.

--- Round 4 ---
-> Infantryman is retreating to save their life!

--- Round 5 ---
-> Infantryman is retreating to save their life!
[State Change]: Infantryman transitions to DefenseState.

--- Round 6 ---
-> Infantryman is in defensive stance, regenerating slightly.
~/Documents/dotnet-labs/lab6  master !28 ?1
```

Висновок: Під час виконання роботи було реалізовано патерн **State**, який дозволив створити гнучкий та адаптивний симулятор бойових дій. Архітектура проєкту забезпечує автоматичну зміну тактики бойової одиниці залежно від подій на полі бою, зберігаючи при цьому чистоту коду та можливість легкого розширення функціоналу в майбутньому.